PDMO - Trabalho Prático: CRUD de Alunos

Objetivo: construir, em React Native, um aplicativo simples de cadastro de alunos que consuma uma API. O foco do trabalho não é criar um layout complexo: é compreender e aplicar os quatro recursos do CRUD usando requisições HTTP.

1. O que o aplicativo deverá fazer

- Cadastrar um aluno informando nome e curso.
- Consultar e exibir a lista de alunos cadastrados.
- Selecionar um aluno para editar seus dados.
- Excluir um aluno.
- Informar ao usuário quando estiver carregando ou quando ocorrer um erro.

2. Recursos que deverão ser utilizados

Recurso	Função no trabalho
useState	Guardar nome, curso, lista de alunos, aluno em edição e estado de carregamento.
useEffect	Executar a consulta inicial quando a tela abrir.
fetch	Fazer a comunicação HTTP com a API.
async / await	Esperar a resposta da API sem travar a lógica do aplicativo.
GET	Consultar a lista de alunos.
POST	Cadastrar um novo aluno.
PUT	Alterar um aluno existente.
DELETE	Excluir um aluno.
response.json()	Converter a resposta JSON recebida da API.
JSON.stringify()	Transformar os dados do aplicativo em JSON antes do envio.
try / catch / finally	Tratar sucesso, erro e encerramento de uma operação.
response.ok	Verificar se a requisição HTTP foi aceita.
FlatList	Exibir a lista de alunos.
TextInput	Receber nome e curso.
Button / Pressable	Executar cadastrar, salvar, editar e excluir.
ActivityIndicator	Indicar que o aplicativo está aguardando a API.
Alert	Exibir mensagens de validação, sucesso ou erro.

3. Modelo de dados

Cada aluno deverá possuir um identificador (id), um nome e um curso. O id será usado principalmente para localizar o registro correto nas operações de alteração e exclusão.

Campo	Exemplo
id	1
nome	João
curso	Informática

4. Adaptação para o laboratório: CRUD local primeiro

Como a rede do laboratório possui restrições de acesso, o trabalho será desenvolvido em duas fases. Na primeira fase, obrigatória e executada no laboratório, os dados ficarão no próprio aplicativo. Assim, será possível praticar CREATE, READ, UPDATE e DELETE sem depender de servidor, porta de rede ou serviço externo.

Nesta fase, utilize uma lista de alunos armazenada com useState. As operações deverão ser realizadas sobre essa lista.

CRUD	No aplicativo local	Recurso principal
CREATE	Adicionar um novo aluno à lista	useState e atualização da lista
READ	Exibir os alunos existentes	FlatList
UPDATE	Localizar o aluno pelo id e alterar seus dados	id, estado e transformação da lista
DELETE	Localizar o aluno pelo id e removê-lo	id e filtragem da lista

Importante: nesta etapa o professor não fornecerá uma URL. O objetivo é dominar a lógica do CRUD antes de acrescentar a comunicação HTTP.

5. Etapa 1 - Montar a interface

Antes do CRUD, monte a tela. Ela deverá possuir:

- Título: Cadastro de Alunos.
- Campo de texto para o nome.
- Campo de texto para o curso.
- Botão para cadastrar/salvar.
- Área para exibir a lista.
- Em cada aluno listado: nome, curso, botão Editar e botão Excluir.

Primeiro confirme que todos os elementos aparecem na tela.

6. Etapa 2 - READ local: exibir os alunos

Crie um estado para armazenar a lista de alunos e utilize FlatList para exibi-la. Comece com alguns registros de teste ou com uma lista vazia, conforme orientação do professor.

Recursos a utilizar: useState, array/lista, FlatList e keyExtractor.

Checkpoint: os alunos guardados no estado devem aparecer corretamente na tela.

7. Etapa 3 - CREATE local: cadastrar

Crie uma função para cadastrar. Antes de inserir um aluno, valide se nome e curso foram preenchidos. O novo aluno deverá receber um id e ser acrescentado à lista existente. Depois do cadastro, limpe os campos.

Recursos a utilizar: if, Alert, useState, objeto JavaScript e atualização da lista.

Checkpoint: o novo aluno deve aparecer na FlatList sem reiniciar o aplicativo.

8. Etapa 4 - DELETE local: excluir

Adicione um botão Excluir em cada item. A função deverá receber o id do aluno e atualizar a lista removendo somente o registro correspondente.

Recurso a pesquisar/utilizar: filter().

Checkpoint: ao excluir, somente o aluno escolhido deve desaparecer da lista.

9. Etapa 5 - UPDATE local: editar

Ao clicar em Editar, carregue o nome e o curso daquele aluno nos TextInput e guarde o id selecionado em um estado. O botão principal deverá mudar de comportamento: em vez de cadastrar, deverá salvar a alteração.

Ao salvar, localize o aluno pelo id, altere seus dados na lista, limpe os campos e saia do modo de edição.

Recursos a pesquisar/utilizar: useState, id do aluno, if/else e map().

Checkpoint: o aluno alterado deve aparecer com os novos dados.

10. Etapa 6 - Validação e mensagens

Use Alert para informar campos vazios e situações importantes. O aplicativo não deverá cadastrar ou salvar um aluno sem nome ou curso.

Desafio: pense em como impedir que uma operação altere o aluno errado. Qual informação identifica cada registro?

11. Fluxo da fase local

```
ABRIR APP
   ↓
READ → FlatList exibe a lista local
   ↓
[Nome] [Curso]
   ↓
CREATE → adicionar aluno
   ↓
EDITAR → preencher campos → UPDATE → salvar
   ↓
EXCLUIR → DELETE
   ↓
LISTA LOCAL ATUALIZADA
```

12. Fase 2 - Relação com uma API e requisições HTTP

Depois de compreender o CRUD local, relacione cada operação com o que aconteceria se os dados estivessem em uma API. Devido às restrições da rede do laboratório, esta parte poderá ser estudada, demonstrada pelo professor ou executada em ambiente apropriado.

CRUD	HTTP	Recursos da versão com API
CREATE	POST	fetch, JSON.stringify(), async/await
READ	GET	fetch, response.json(), async/await
UPDATE	PUT	fetch, id, JSON.stringify(), async/await
DELETE	DELETE	fetch, id, async/await

Na versão com API também aparecem response.ok, try/catch/finally e ActivityIndicator. O objetivo é perceber que a função do CRUD continua a mesma; muda o local onde os dados são armazenados.

13. Como seria o fluxo com API

```
ABRIR APP
   ↓
GET → listar alunos
   ↓
[Nome] [Curso]
   ↓
POST → cadastrar
   ↓
EDITAR → preencher campos → PUT → salvar
   ↓
EXCLUIR → DELETE
   ↓
GET → atualizar lista
```

14. Requisitos para entrega

- O aplicativo deve executar sem erro.
- As quatro operações CRUD locais devem funcionar.
- Nome e curso devem ser validados antes de cadastrar/salvar.
- O código deve utilizar funções separadas para as operações principais.
- Cada aluno deve possuir um id.
- O aluno deverá conseguir explicar Create, Read, Update e Delete.
- O aluno deverá conseguir relacionar CREATE/READ/UPDATE/DELETE com POST/GET/PUT/DELETE em uma API.

15. Desafio final de compreensão

Explique: qual é a diferença entre CREATE, READ, UPDATE e DELETE no aplicativo local? Depois, relacione cada operação com GET, POST, PUT e DELETE. Para a versão com API, explique também para que servem JSON.stringify(), response.json() e async/await.